

PROJECT REPORT 3

Development Process, Honest Reflection, and Project Journey

Project Title:

Event-Driven CSV Data Ingestion and Graph-Based Chatbot using FastAPI, Apache Kafka, and Neo4j

1. Introduction

This report documents the complete development journey of our project. It explains how the idea was transformed into a working system, the technical challenges encountered during implementation, the engineering decisions made, and the lessons learned throughout the hackathon. Rather than focusing only on the final outcome, this report presents an honest reflection on the development process and highlights areas for future improvement.

2. Project Vision

The primary goal of the project was to build an automated platform capable of processing dynamic CRM datasets and enabling users to interact with the uploaded data using natural language.

Many organizations receive operational data in CSV format. Extracting meaningful information from these files usually requires manual database imports and technical knowledge of query languages. Our objective was to simplify this process by automating data ingestion and providing an intuitive chatbot interface.

3. Development Timeline

Phase Activities

Phase 1 Problem statement analysis and requirement gathering

Phase 2 Technology selection and architecture design

Phase 3 Frontend development

Phase 4 FastAPI backend implementation

Phase 5 Kafka producer and consumer integration

Phase 6 Neo4j graph database integration

Phase 7 Chatbot and Cypher query implementation

Phase 8 Testing, debugging, and documentation

4. Development Process

Phase 1 - Requirement Analysis

The first step was understanding the hackathon problem statement. The team analyzed the expected workflow, identified the required technologies, and defined the project scope.

Key observations included:

Dynamic CSV upload support.

Event-driven architecture.

Graph database integration.

Conversational data retrieval.

Modular software design.

Phase 2 - Architecture Design

Before implementation, a complete system architecture was prepared.

The objective was to ensure that every module performed a single responsibility while remaining independent from the others.

Major architectural components included:

Frontend

FastAPI Backend

Dataset Processor

Kafka Producer

Kafka Consumer

Neo4j Database

Chat Service

This modular approach simplified development and future maintenance.

Phase 3 - Frontend Development

A lightweight web interface was developed to provide two primary functions:

CSV Upload

Chat Interface

The interface was intentionally kept simple to ensure usability and quick demonstration during the hackathon.

Phase 4 - Backend Development

FastAPI was selected as the backend framework because of its high performance and

ease of API development.

The backend was responsible for:

Receiving uploaded CSV files.

Reading datasets using Pandas.

Validating uploaded files.

Sending rows to Kafka.

Handling chatbot requests.

Communicating with Neo4j.

Returning responses to users.

Phase 5 - Kafka Integration

Apache Kafka was introduced as the messaging layer between the backend and Neo4j.

Instead of directly writing data into the database, every CSV row was published as an independent Kafka message.

The Kafka Consumer continuously listened for incoming messages and transferred them to Neo4j.

This separation improved reliability and scalability.

Phase 6 - Neo4j Integration

Neo4j was configured as the primary database.

Instead of storing information in relational tables, the system represented the uploaded CRM data using graph nodes and relationships.

The Loader used the MERGE command to prevent duplicate nodes and maintain graph consistency.

Phase 7 - Chatbot Development

The chatbot was designed as a retrieval-based system.

Instead of generating information, it retrieves answers from Neo4j.

Workflow:

Receive the user question.
Convert it into a Cypher query.
Execute the query.
Retrieve matching records.
Return the final response.

This ensured that responses were based only on the uploaded dataset.

5. Challenges Faced

Throughout the development process, several technical challenges were encountered.

Challenge 1 - Kafka Configuration

Initially, configuring Kafka and establishing communication between the producer and consumer required careful setup.

Solution

A dedicated Kafka topic was created, and producer/consumer responsibilities were clearly separated.

Challenge 2 - Neo4j Data Modeling

Converting tabular CSV data into graph structures required careful planning.

Solution

Graph nodes and relationships were designed before implementation to ensure consistency.

Challenge 3 - Duplicate Records

Repeated uploads could create duplicate nodes.

Solution

The MERGE operation was used instead of CREATE during graph insertion.

Challenge 4 - API Integration

Connecting the frontend, backend, Kafka, and Neo4j required proper API communication.

Solution

REST APIs were designed with clear request and response formats.

Challenge 5 - Dataset Quality

Uploaded CSV files contained inconsistent formatting.

Solution

Dataset validation and cleaning steps were introduced before ingestion.

6. Testing Process

The completed system was tested using multiple scenarios.

Testing included:

Valid CSV upload

Invalid CSV upload

Empty dataset

Duplicate records

Missing values

Chat queries

Neo4j insertion

Kafka message delivery

API responses

Each module was tested individually before performing complete system integration.

7. Lessons Learned

This project provided practical experience in several engineering concepts.

Event-Driven Architecture

Understanding how asynchronous communication improves scalability.

REST API Development

Designing backend services using FastAPI.

Apache Kafka

Learning producer-consumer communication.

Graph Databases

Understanding nodes, relationships, and graph traversal.

Cypher Query Language

Writing graph queries instead of SQL.

Docker

Running multiple services in isolated containers.

System Integration

Combining frontend, backend, messaging systems, and databases into a complete solution.

8. Honest Reflection

Several objectives were successfully achieved during development.

Completed features:

Dynamic CSV upload

Dataset validation

Dataset cleaning

Kafka integration

Neo4j integration

Graph storage

Chatbot interface

Cypher-based retrieval

Docker deployment

Areas that require further improvement:

Better natural language understanding

Authentication and user management

- Advanced graph visualization
- Multi-file support
- Cloud deployment
- Performance optimization for very large datasets

This reflection represents the current state of the project and identifies realistic opportunities for enhancement.

9. Future Roadmap

The project can be extended with several advanced capabilities.

Future improvements include:

- Excel and JSON file support
- PDF document ingestion
- Role-based authentication
- Cloud deployment using Kubernetes
- AI-assisted query interpretation
- Interactive analytics dashboard
- Real-time monitoring
- Multi-user collaboration
- Automated schema detection
- Graph visualization tools

These additions would improve usability, scalability, and enterprise readiness.

10. Team Contribution

The project was completed through collaboration across multiple development areas.

Key responsibilities included:

- Requirement analysis
- Architecture design
- Frontend development
- Backend implementation
- Kafka integration
- Neo4j integration
- Testing and debugging
- Documentation

Each phase contributed to the successful completion of the system.

11. Conclusion

The project successfully demonstrates how event-driven architecture can simplify data ingestion and retrieval from dynamic datasets. By combining FastAPI, Apache Kafka, Neo4j, and Cypher, we developed a modular system capable of processing uploaded CSV files and providing conversational access to the stored information.

The development process highlighted the importance of modular software design, reliable messaging systems, graph databases, and effective API integration. Although there are opportunities for future enhancement, the current implementation provides a scalable and maintainable solution that satisfies the primary objectives of the hackathon problem statement.

Final Reflection

This project was not only an implementation exercise but also a learning experience in designing distributed systems. It strengthened our understanding of event-driven architectures, graph databases, backend development, and system integration. The experience gained during this project provides a strong foundation for building larger, enterprise-scale data processing applications in the future.